API SECURITY

RISK ANALYSIS REPORT

Presented By:

Pranav Suresh

Cyber Security Intern

Future Interns

Date: 22 April 2026

1. Introduction

This report presents the results of an API Security Risk Analysis conducted on publicly accessible REST APIs to identify potential security vulnerabilities. APIs (Application Programming Interfaces) are critical components of modern web applications that allow systems to communicate with each other. Insecure APIs can expose sensitive data, allow unauthorized access, and become entry points for attackers.

The assessment aims to evaluate the security posture of the selected APIs and highlight vulnerabilities that could be exploited by malicious actors. The findings of this report will help in understanding the security gaps and improving the overall protection of API-based systems.

2. Objective

The primary objective of this assessment is to identify security vulnerabilities present in the target APIs, analyze the associated risks, and provide suitable recommendations to mitigate these security issues. The goal is to enhance the overall security and reliability of API-based applications.

3. Scope of Assessment

The scope of this assessment is limited to the following publicly accessible test APIs:

- Target API 1: https://jsonplaceholder.typicode.com
  A free fake REST API used for testing and prototyping.

- Target API 2: https://reqres.in
  A hosted REST API designed for testing front-end applications.

The assessment focuses on identifying common API security vulnerabilities using manual testing techniques. No destructive or intrusive testing was performed during this assessment.

4. Tools Used

- Postman – Used to send API requests, analyze responses, and test various API endpoints for security vulnerabilities.

- Browser DevTools – Used to inspect API calls, headers, and responses directly from the browser.

5. Methodology

The API security assessment was conducted using a structured

approach involving manual testing of API endpoints.

- API Endpoint Testing
  Each API endpoint was tested by sending various HTTP requests including GET and POST methods using Postman. The responses were analyzed for security issues such as unauthorized data access, missing authentication, and verbose error messages.

- Input Validation Testing
  Invalid and empty inputs were submitted to API endpoints to test how the API handles unexpected data and whether it reveals sensitive internal information in error responses.

- Access Control Testing
  API endpoints were tested by modifying request parameters such as user IDs to check whether unauthorized access to other users' data was possible.

- Rate Limiting Testing
  Repeated requests were sent to API endpoints to check whether any rate limiting or throttling mechanisms were in place to prevent abuse.

6. Findings and Analysis

i. No Authentication & Sensitive Data Exposure

- **Risk Level: High**
- Description: The API endpoint at https://jsonplaceholder.typicode.com/users returns complete user profile information including names, email addresses, phone numbers, physical addresses and GPS coordinates without requiring any authentication or API key. Any unauthorized user can freely access this sensitive data without providing any credentials.

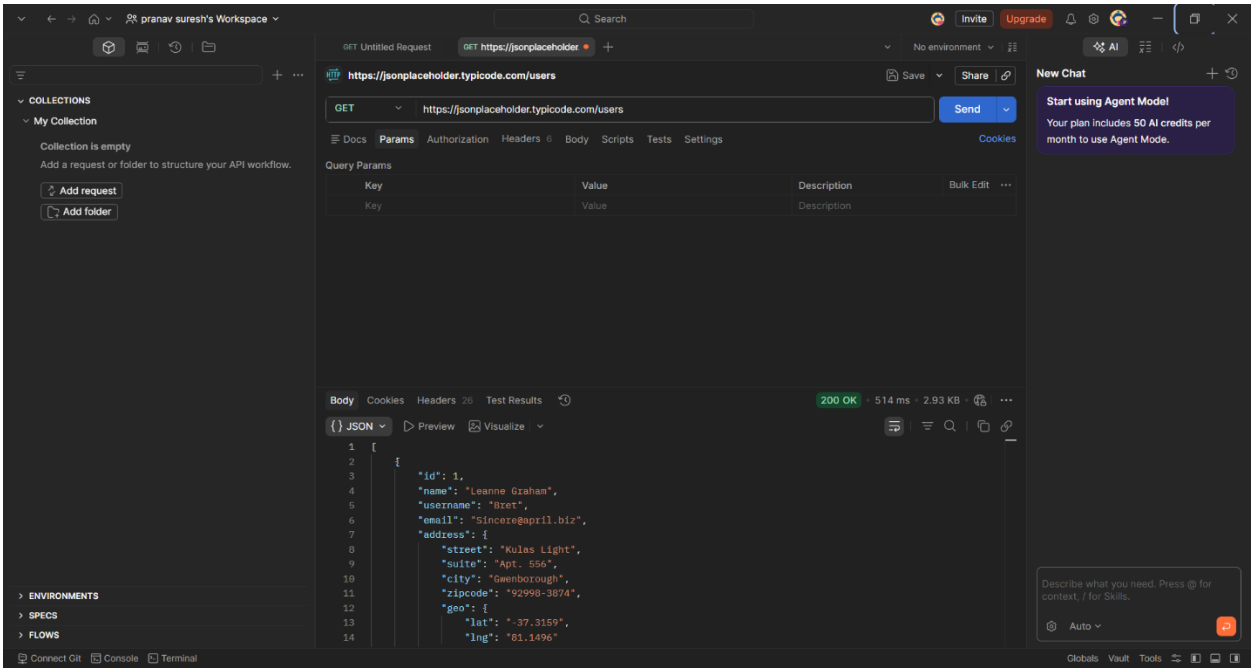

Figure 1: API returning sensitive user data without authentication (User 1)

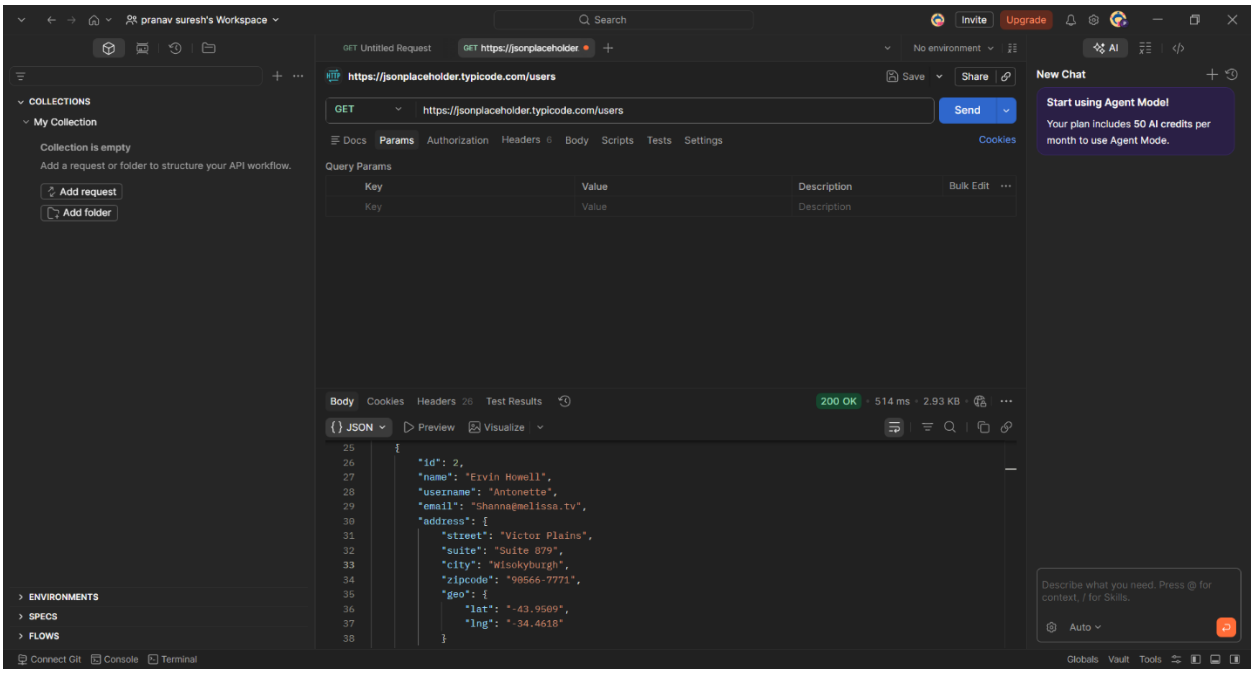

Figure 2: API returning sensitive user data without authentication (User 2)

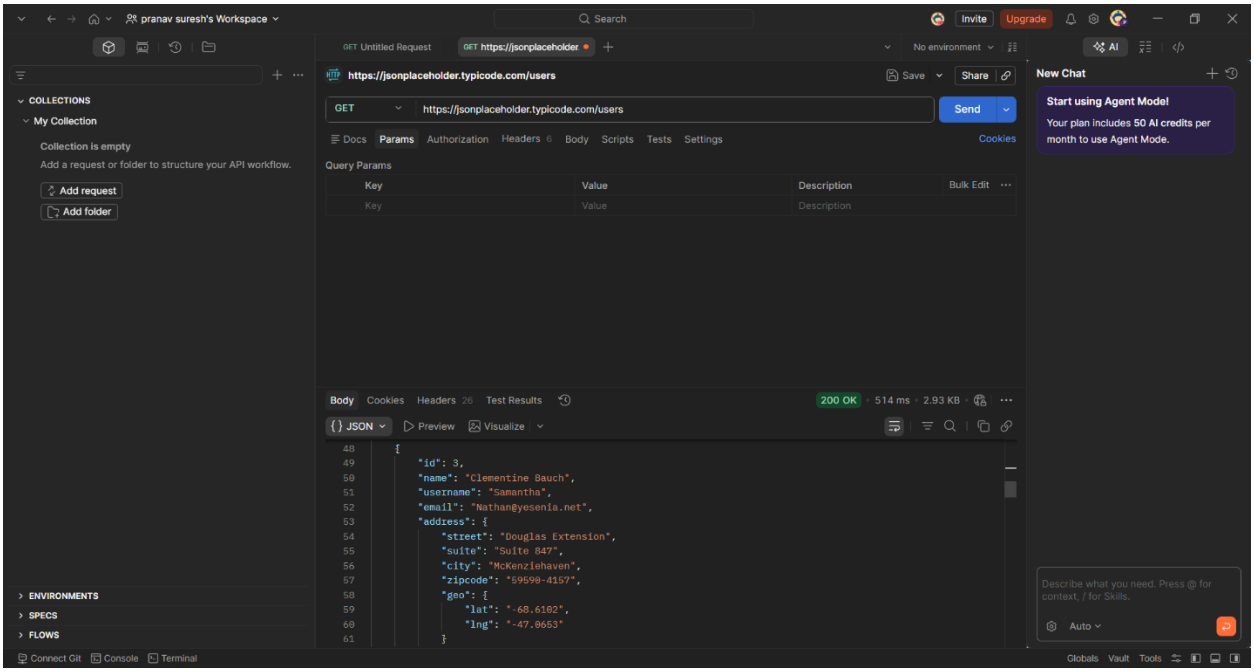

Figure 3: API returning sensitive user data without authentication (User 3)

- Recommendation: Implement authentication mechanisms such as API keys or OAuth tokens to restrict access to sensitive endpoints. Ensure that only authorized users can retrieve personal data.

ii. Broken Access Control

- Risk Level: High

- Description: The API allows access to any individual user's data by simply modifying the user ID in the URL. Endpoints such as /users/1, /users/2, and /users/3 return complete profile information of different users without any authorization check. An attacker can enumerate through user IDs to access any user's personal information including their name, email, phone number, home address and GPS location coordinates.

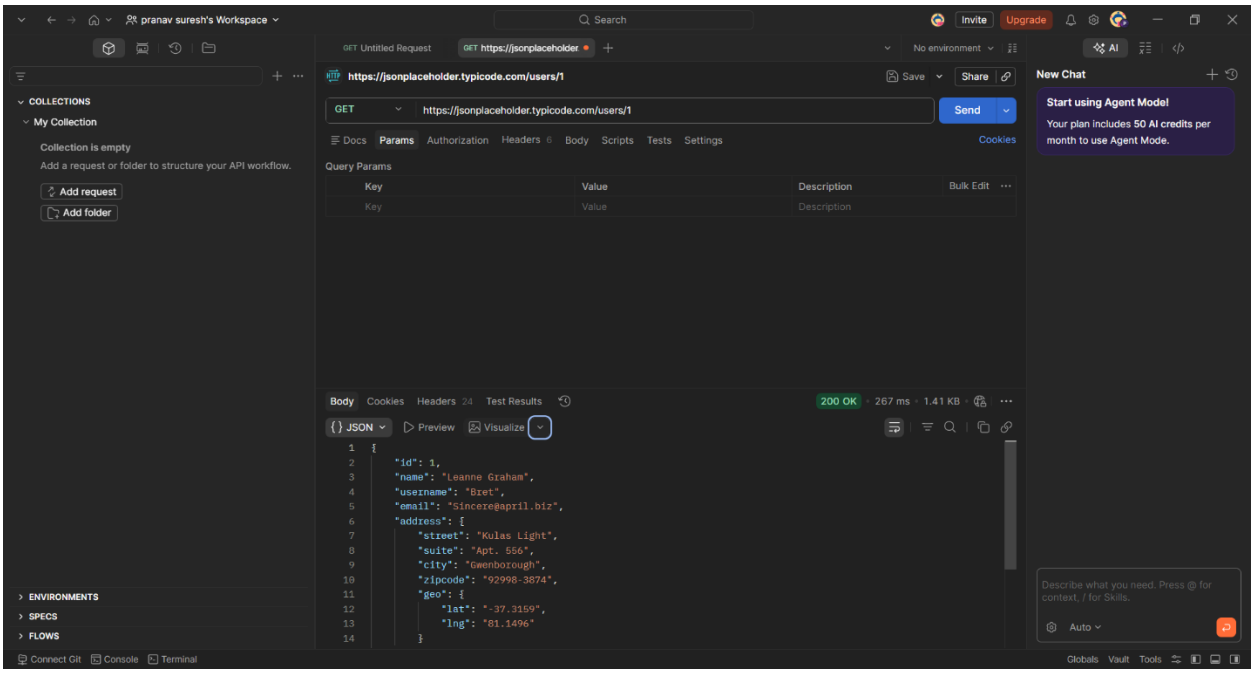

Figure 4: Accessing User 1 data by modifying ID in URL

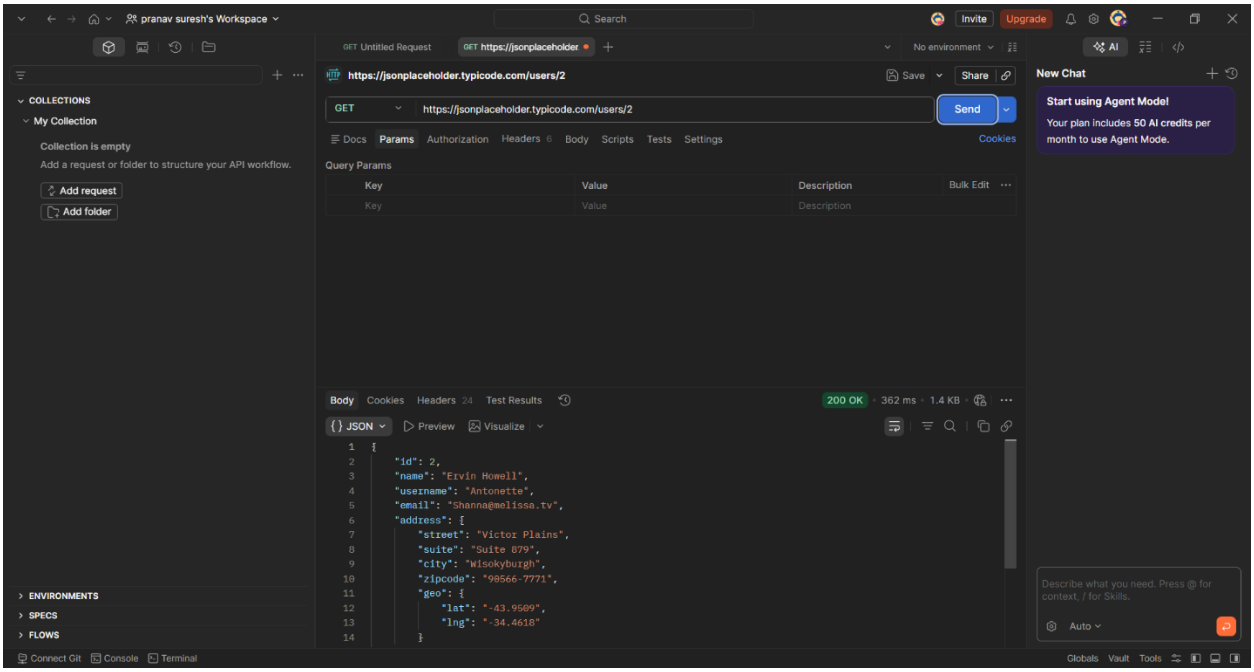

Figure 5: Accessing User 2 data by modifying ID in URL

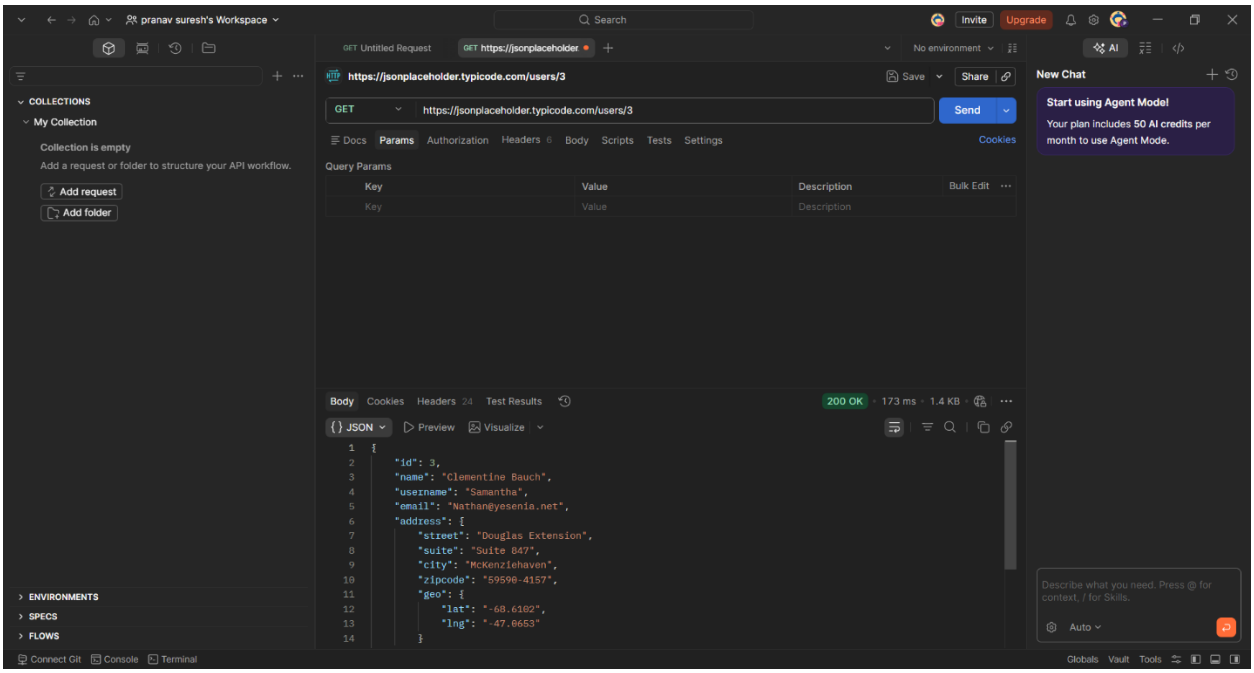

Figure 6: Accessing User 3 data by modifying ID in URL

- Recommendation: Implement proper authorization checks on all API endpoints. Ensure users can only access their own data. Use non-sequential and non-predictable resource identifiers such as UUIDs instead of simple numeric IDs.

iii. No Rate Limiting

- Risk Level: Medium

- Description: The API does not implement any rate limiting or request throttling mechanisms. Repeated requests to the same endpoint continuously return 200 OK responses with no restrictions or blocking. A secure API should return a

429 Too Many Requests error after a certain number of requests within a time period. The absence of rate limiting makes the API vulnerable to brute force attacks and denial of service attacks.

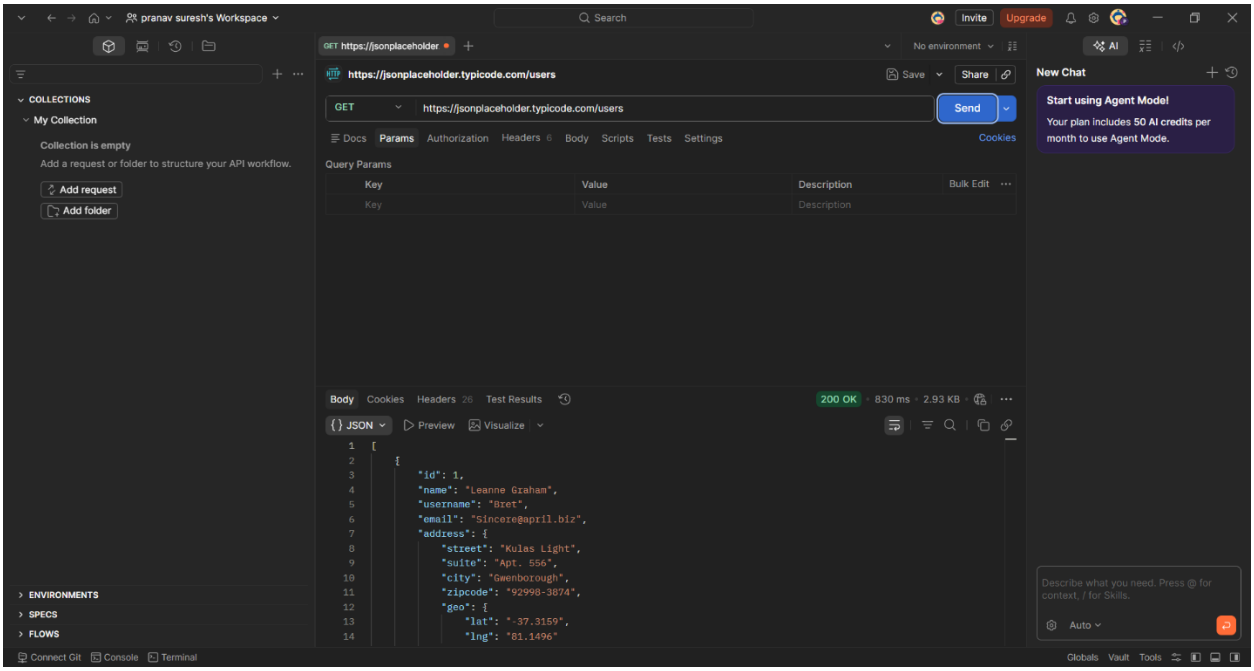

Figure 7: First request returning 200 OK

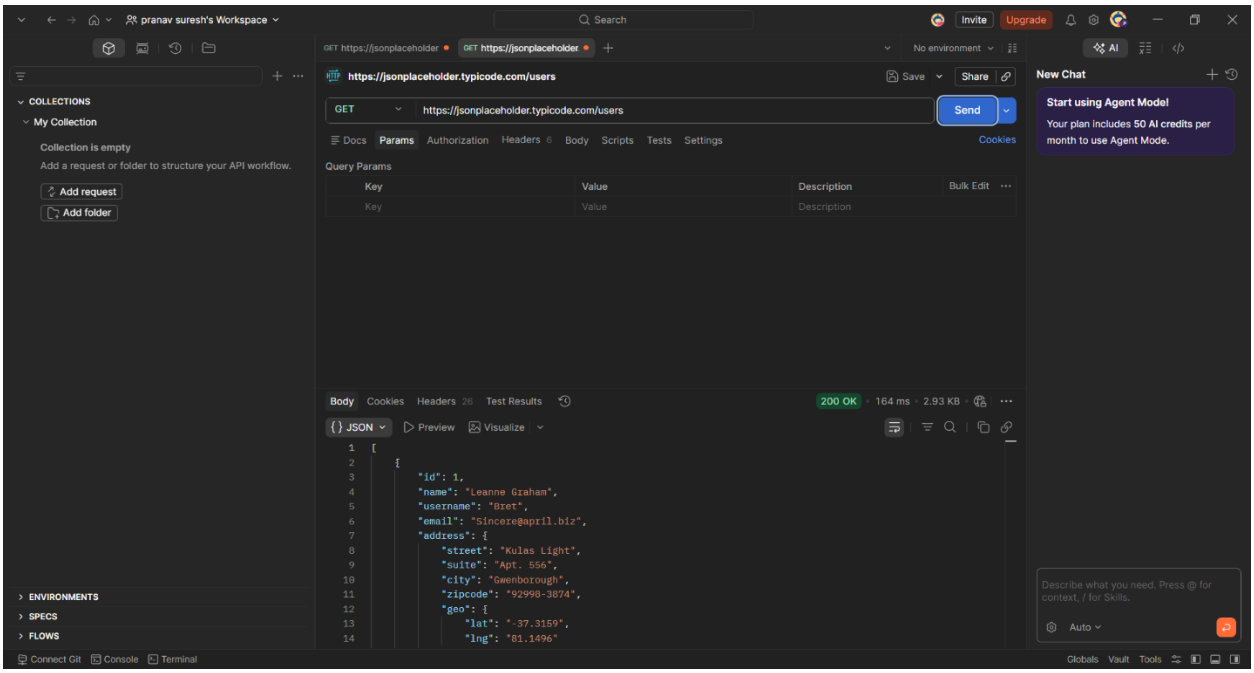

Figure 8: Repeated requests still returning 200 OK with no rate limiting

- **Recommendation**: Implement rate limiting on all API endpoints to restrict the number of requests a client can make within a given time period. Return a proper 429 Too Many Requests response when the limit is exceeded.

iv. Verbose Error Messages

- **Risk Level: High**
- **Description:** When an unauthorized request is sent to the API endpoint at https://reqres.in/api/login, the API returns a detailed error response revealing sensitive internal information. The error message exposes the exact

authentication mechanism required, internal endpoint structure, header names and formats, and even provides example commands showing exactly how to authenticate. This information can be used by an attacker to understand the API structure and craft targeted attacks.

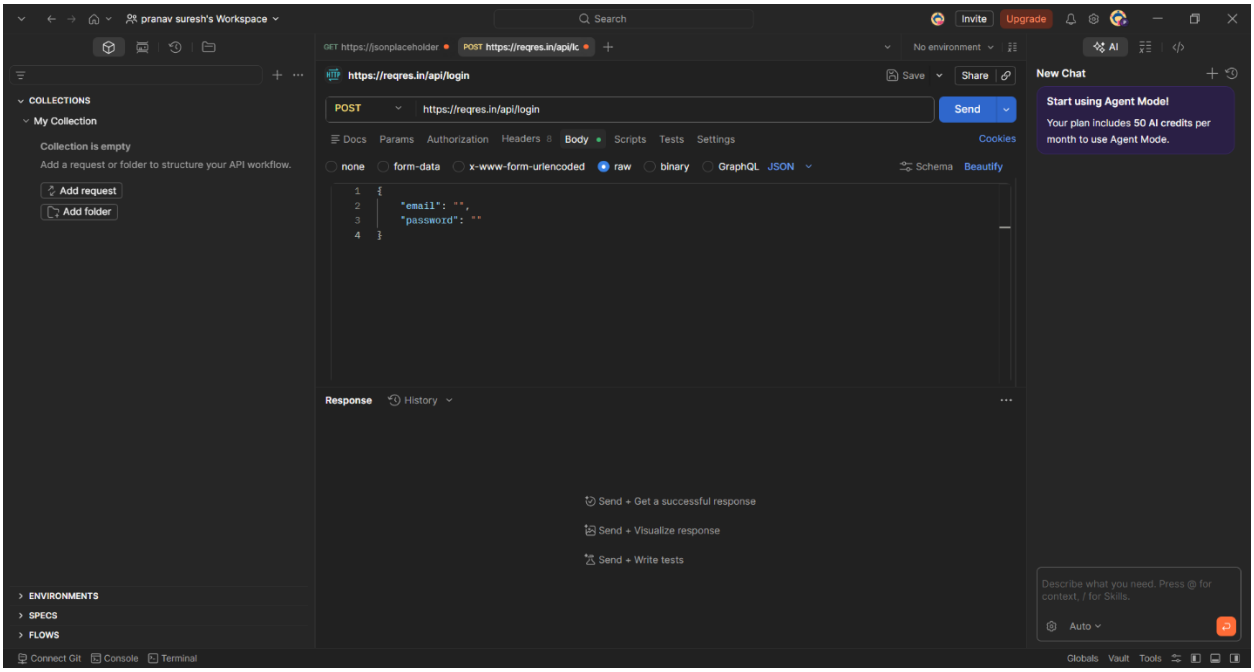

Figure 9: Request body sent to login endpoint

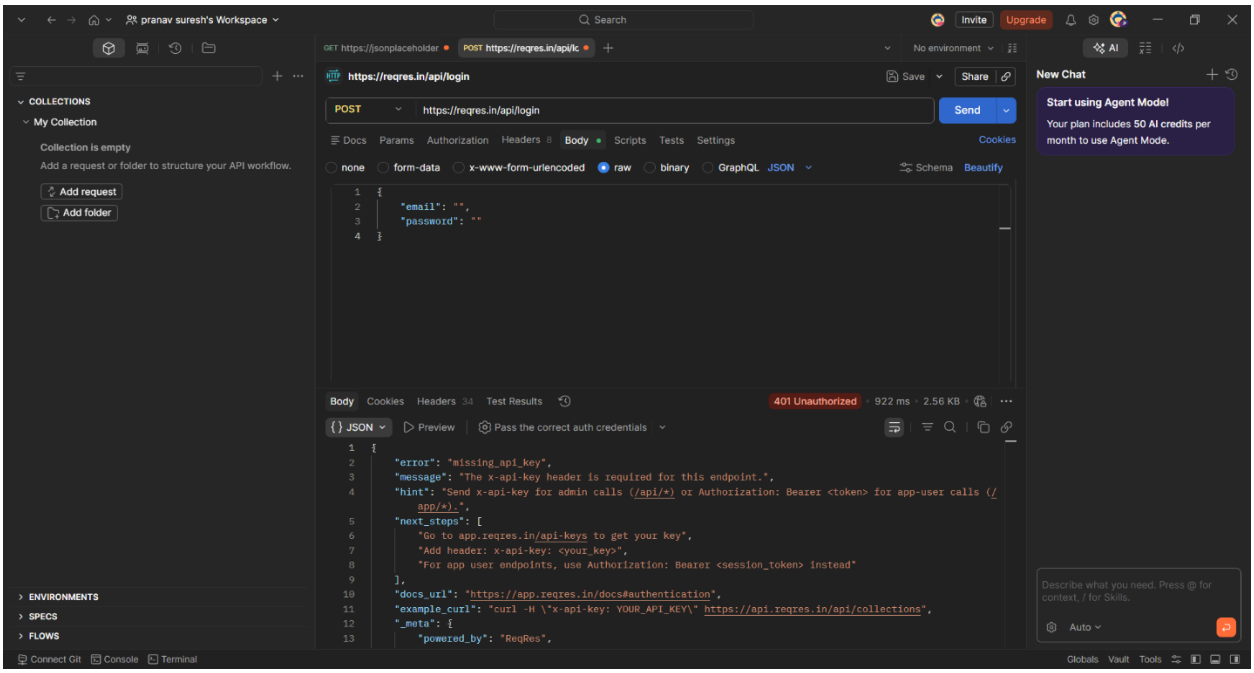

Figure 10: Verbose error response revealing internal API details

- **Recommendation:** Implement generic error messages that do not reveal internal system details. Error responses should only return necessary information such as an error code and a simple message without exposing authentication mechanisms or internal structure.

v. Missing Input Validation

- Risk Level: High

- Description: The API endpoint at https://reqres.in/api/register accepts requests containing invalid email formats and empty

password fields without proper validation. When invalid data is submitted the API returns error responses that confirm the existence of the endpoint and reveal expected input formats. Proper input validation should reject malformed requests before processing them and return minimal information in the error response.

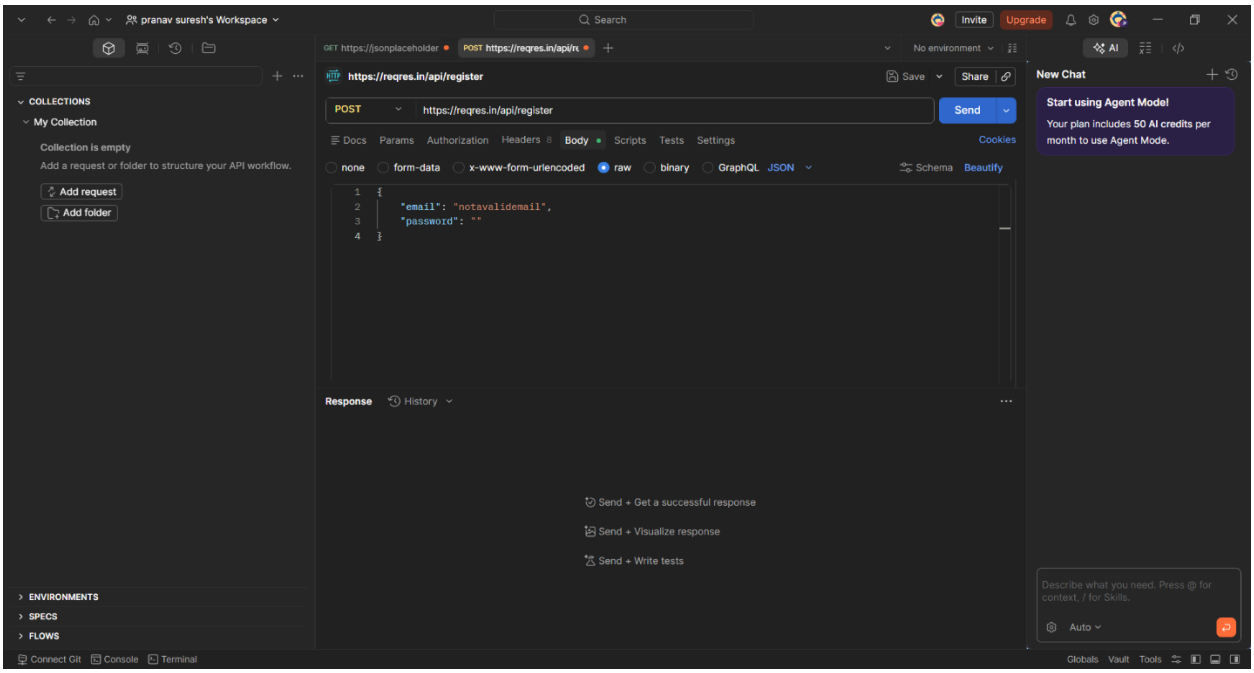

Figure 11: Invalid input data sent in request body

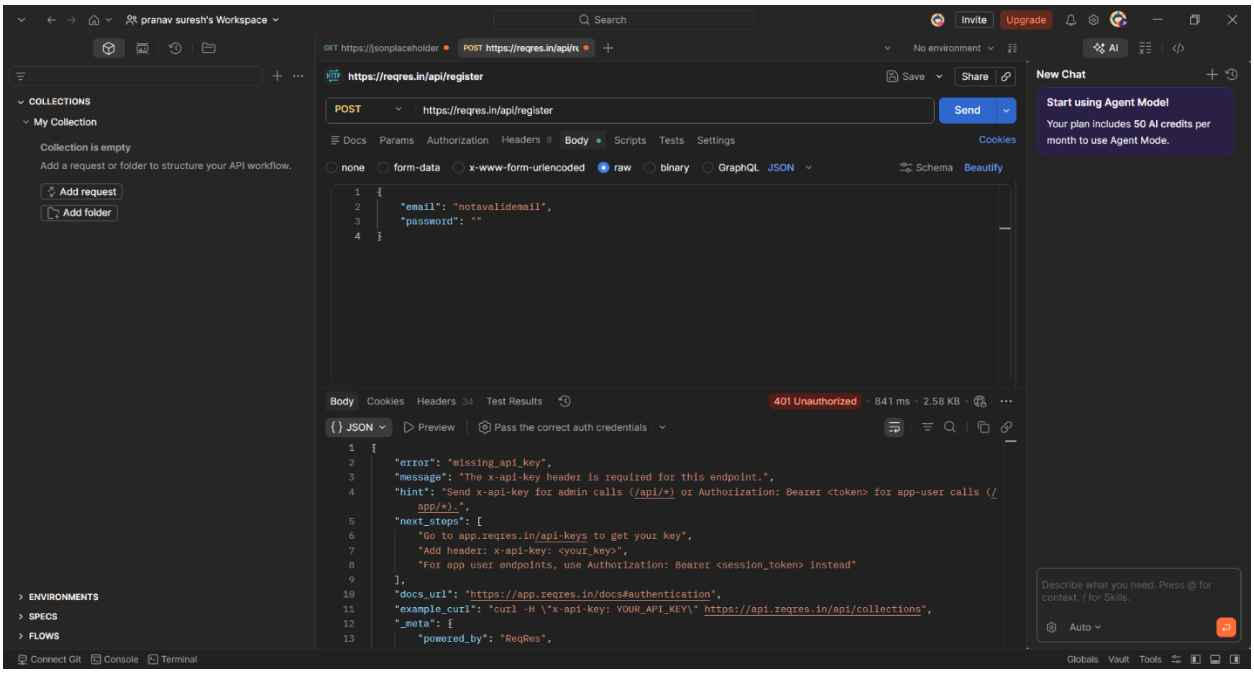

Figure 12: API response showing missing input validation

- **Recommendation:** Implement strict input validation on all API endpoints. Validate email formats, enforce password requirements, and reject requests with empty or invalid fields before processing. Return generic error messages that do not confirm field names or expected formats.

7. Risk Classification

The identified vulnerabilities were classified based on their severity and potential impact.

- **High Risk:** Severe vulnerabilities that can be easily

exploited and cause major damage including unauthorized data access and privacy breaches.

- **Medium Risk:** Vulnerabilities that may require certain conditions but can still significantly impact the security of the API.

- **Low Risk:** Vulnerabilities with limited impact but should still be addressed to maintain good security practices.

#	Vulnerability	API Tested	Risk Level
i	No Authentication & Sensitive Data Exposure	jsonplaceholder.typicode.com	High
ii	Broken Access Control	jsonplaceholder.typicode.com	High
iii	No Rate Limiting	jsonplaceholder.typicode.com	Medium
iv	Verbose Error Messages	reqres.in	High
v	Missing Input Validation	reqres.in	High

8. Recommendations

Based on the findings, the following recommendations are suggested:

- Implement authentication mechanisms such as API keys or OAuth tokens on all sensitive endpoints.

- Enforce proper authorization checks to ensure users can only access their own data.

- Implement rate limiting to prevent brute force and denial of service attacks.

- Replace verbose error messages with generic responses that do not reveal internal system details.

- Implement strict input validation on all API endpoints to reject malformed or empty requests.

- Perform regular API security assessments to identify and address new vulnerabilities.

- Use HTTPS for all API communications to ensure data is encrypted in transit.

9. Conclusion

The assessment identified five security vulnerabilities across the tested APIs including issues related to authentication, access control, rate limiting, error handling and input validation. Three vulnerabilities were classified as High Risk and two as Medium Risk. Implementing the recommended measures will significantly improve the overall security posture of API-based applications and reduce the risk of unauthorized access and data breaches.